Imov.Plan

Planejamento inteligente e simulação completa para aquisição imobiliária

DOCUMENTAÇÃO TÉCNICA

Arquitetura, regras de negócio e guia de operação

Visão geral do sistema, modelo de dados, motor de cálculo, referência da API, segurança, integrações, frontend, deploy e pontos de atenção.

.NET 9 · ASP.NET Core · MongoDB · Next.js 16 · React 19 · Tailwind CSS 4 · Zustand · React Query · Docker · Vercel · PWA

REPOSITÓRIO
MatheusRafaDev/imov-plan

COMMIT ANALISADO
58d3932

GERADO EM
25/10/2026

Imov.Plan — Documentação Técnica Completa

> Plataforma web de planejamento financeiro para compra de imóvel: calcula quanto tempo leva para juntar a entrada (e os custos de compra), considerando rendimento (CDI), IR regressivo, aportes extras, retiradas e múltiplos participantes. Também simula o financiamento (SAC/Price), calcula o CET e oferece consultoria por IA.
>
> Repositório: https://github.com/MatheusRafaDev/imov-plan Stack: **.NET 9 (ASP.NET Core) + MongoDB** no backend, **Next.js 16 + React 19 + Tailwind 4** no frontend. Documento gerado a partir da leitura do código-fonte (commit `58d3932`). Pontos que divergem do README original estão marcados em Divergências e pontos de atenção.

Sumário

1. Visão geral
2. Arquitetura
3. Estrutura do repositório
4. Modelo de dados (MongoDB)
5. Regras de negócio e motor de cálculo
6. Referência da API REST
7. Autenticação e segurança
8. Integrações externas
9. Frontend
10. Configuração e variáveis de ambiente
11. Execução local, build e deploy
12. Testes
13. Fluxos principais (ponta a ponta)
14. Divergências e pontos de atenção
15. Glossário

---

1. Visão geral

Problema

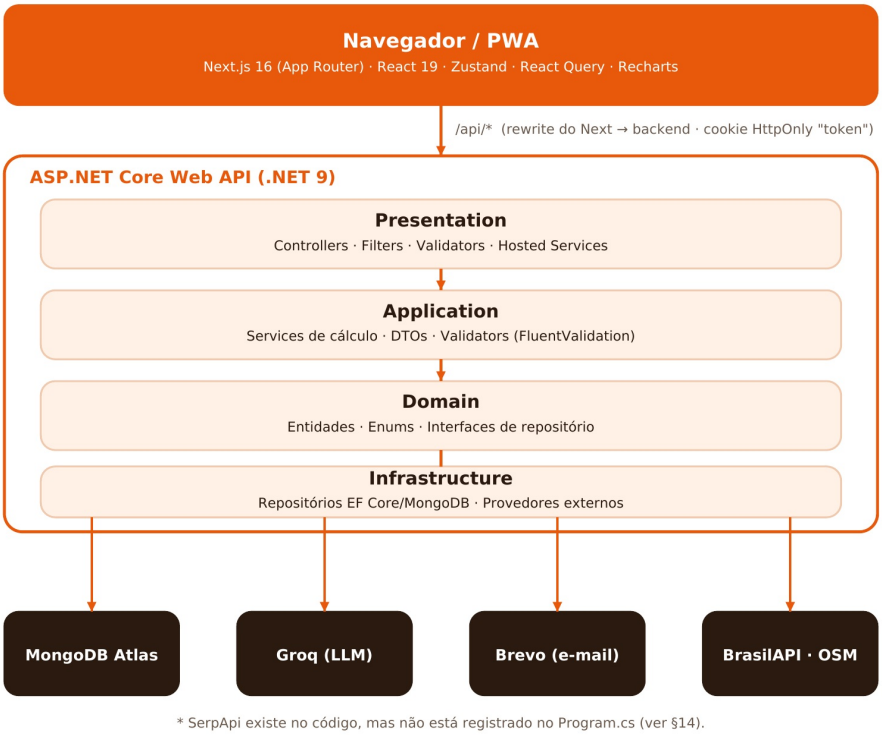
Simuladores bancários calculam só a parcela teórica. O Imov.Plan cobre a **fase anterior à compra**: quanto guardar, por quanto tempo, com que rendimento, e quais custos cartorários entram na conta — por uma ou mais pessoas (casal, família, sócios).

Funcionalidades

Módulo	O que faz

Imóvel	Define valor, % de entrada, % de custos extras, estado/cidade (ITBI), data de início, prazo máximo, CDI e tipo de investimento.
Pessoas	Cadastra participantes: renda, renda complementar, gastos (simples ou detalhados por categoria), aporte mensal, dia do aporte, patrimônio inicial e tipo de investimento.
Planejamento	Aportes extras (FGTS, 13º, bônus, restituição de IR, venda de bens) e retiradas; edição do aporte de meses específicos; cenários pessimista/realista/otimista.
Resultado	Resumo, gráfico de evolução, comparação de cenários, tabela mês a mês, progresso por participante, consultoria por IA.
Lançar aporte	Marca meses como concluídos e registra o saldo real (checkpoint) para recalibrar a projeção.
Financiamento	Comparação SAC × Price, CET, regra dos 30% da renda, simulação de FGTS. Fluxos "Imóvel pronto" e "Imóvel na planta".
Arredores	Pontos de interesse (mercado, farmácia, escola, padaria, parque, hospital) via OpenStreetMap + avaliação da região por IA.
Meus planos	Múltiplos planos por usuário, renomear, excluir, comparar.
Conta	Cadastro/login (e-mail+senha e Google), recuperação de senha, perfil, exportação de dados (LGPD) e exclusão de conta.
PWA	Instalável, com service worker e página offline.

2. Arquitetura



* SerpApi existe no código, mas não está registrado no Program.cs (ver §14).

Estilo: Clean Architecture / DDD leve. Dependências apontam para dentro: `API → Application → Domain` e `Infrastructure → Application/Domain`.

Decisões de design relevantes

- **Simulação calculada no servidor.** O backend é a fonte da verdade; o frontend consome o resultado via seletores (`simulacaoSelectors.ts`). Há testes de **paridade** entre o motor TypeScript e o C# (§12).
- **Autosave de rascunho (draft).** O plano é salvo continuamente (debounce de 1 s) via `PUT /api/plano/draft/{id}`, permitindo trocar de dispositivo sem perder contexto.
- **Só a última simulação é mantida.** Cada recálculo apaga as simulações anteriores do plano e grava uma nova (versão 1).
- **Persistência normalizada em coleções**, mas o DTO de draft devolve tudo agregado em um único objeto.
- **Proxy de API no frontend.** No browser, `baseURL` vira `/api` (reescrito pelo Next) para evitar bloqueio de cookies de terceiros e CORS.

---

3. Estrutura do repositório

```
imov-plan/
├── backend/
│   ├── ImovPlan.slnx
│   ├── Dockerfile                # build local/Docker Compose (porta 8080)
│   ├── Dockerfile.vercel         # build para Vercel (PORT dinâmico, healthcheck)
│   ├── .env.example
│   ├── Core/
│   │   ├── ImovPlan.Domain/            # Entities, Enums, Interfaces (repositórios)
│   │   └── ImovPlan.Application/       # Services, DTOs, Validators
│   ├── Infrastructure/
│   │   └── ImovPlan.Infrastructure/    # AppDbContext, Repositories, Services externos
│   ├── Presentation/
│   │   └── ImovPlan.API/               # Controllers, Program.cs, Filters, hosted services
│   └── Tests/ImovPlan.Application.Tests/   # xUnit + Moq (net10.0)
├── frontend/
│   ├── src/app/                  # Rotas (App Router)
│   ├── src/components/           # UI e componentes de domínio
│   ├── src/context/              # AuthContext, PlanContext (compat.)
│   ├── src/hooks/                # usePlanLogic, usePlanDraft, usePlanos, useSimulacao
│   ├── src/lib/                  # api.ts (axios), finance.ts (motor), seo.ts
│   ├── src/services/             # Wrappers de API
│   ├── src/store/usePlanStore.ts  # Zustand (persistido)
│   ├── src/utils/                # itbi, datas, formatadores, seletores
│   ├── public/                   # manifest, sw.js, offline.html, ícones
│   ├── scripts/                  # gerar-casos-paridade.ts, generate-icons.mjs
│   └── tests/fixtures/casos-paridade.json
├── docker-compose.yml            # backend:5179→8080, frontend:3000
├── docker-compose-updated.yml
├── Dockerfile.vercel             # build do backend a partir da raiz
├── vercel.json                   # monorepo: serviços "web" e "backend"
└── version.txt                   # hash do commit exposto em /api/version
```

---

4. Modelo de dados (MongoDB)

O acesso usa **EF Core com o provider** **`MongoDB.EntityFrameworkCore`** (`AppDbContext`), com `AutoTransactionBehavior = Never`. A convenção global `IgnoreExtraElements` evita falhas ao ler documentos legados.

Coleções

Coleção	Entidade	Observações
`usuarios`	`Usuario`	Índice único (esparso) em `email`.
`planejamentos`	`Planejamento`	Índices: `usuarioId`, `sessionId`, `usuarioId+createdAt↓`. Subdocumento `CustosCompra`.
`participantes`	`Participante`	Índice `planejamentoId`. Subdocumento `PatrimonioInicial`.
`gastosDetalhados`	`GastoDetalhado`	Índices `planejamentoId`, `participanteId`.
`aportesExtras`	`AporteExtra`	Índice `planejamentoId`. Aportes e retiradas.
`historicoAportes`	`HistoricoAporte`	Edições manuais do aporte regular (mês → valor). Índice composto.
`historicoSimulacoes`	`HistoricoSimulacao`	Última simulação + `ParticipantesSnapshot`.
`evolucaoMensalSimulacoes`	`EvolucaoMensalSimulacao`	Linha por mês da simulação (`simulacaoId+mes`).
`parametrosFinanceiros`	`ParametrosFinanceiros`	Tabelas de taxas e custos (§5.5). Índice `codigo`.
`pontosInteresseCache`	`PontoInteresseCache`	**TTL de 15 dias** em `createdAt`.

Os índices são criados na inicialização por `MongoDbIndexInitializer` (hosted service); o TTL do cache é criado direto no `Program.cs`.

Entidades principais

Usuario — `Id, Nome, Email, PasswordHash, DataNascimento?, TipoInvestimento?, Role?, Provider?, ResetPasswordToken?, ResetPasswordExpiry?, ResetPasswordTokenUsed?, CreatedAt` `Provider` ∈ `local | google | both`. `Role` ∈ `User | Admin`.

Planejamento — `Id, NomePlano, ValorImovel?, PercentualEntrada?, PercentualCustosExtras?, ValorJaGuardado?, PrazoMaxMeses?, TaxaCdiAnual?, PercentualCdi?, DataInicio?, BancoEscolhidoId/Nome/Taxa?, ParticipantesIds[], MesesConcluidos[], UsuarioId?, SessionId?, TipoInvestimento?, Estado?, Cidade?, Status ("Draft"|"Completed"), CustosCompra?, CreatedAt` `CustosCompra` = `{ ValorEntrada, CustoITBI, CustoEscritura, CustoRegistro, TotalNecessario, CalculadoEm }`.

Participante — `Id, PlanejamentoId, Nome, RendaMensal, RendaComplementar, GastosMensais, UsarGastosDetalhados, SobraMensal, AporteMensal, DiaAporte? (padrão 5), PatrimonioInicial?, ValorAtual?, DataValorAtual?, CreatedAt` `PatrimonioInicial` = `{ Valor, Fonte (enum FonteSaldo), TipoInvestimento?, RegistradoEm }`. `FonteSaldo`: `Poupanca, ContaCorrente, RendaFixa, RendaVariavel, Investimento, Outros`. `ValorAtual/DataValorAtual` formam um **checkpoint**: o saldo real informado pelo usuário, que substitui o saldo teórico.

AporteExtra — `Id, PlanejamentoId, ParticipanteId?, Valor, Data, Origem, Tipo ("Aporte"|"Retirada"), Motivo?, Revertido?, RevertidoEm?, CreatedAt`. Itens `Revertido = true` são ignorados na simulação.

HistoricoSimulacao — entradas (`ValorImovel, TotalNecessario, ValorJaGuardado,`

`AporteMensalTotal, TaxaCdiAnual, PercentualCdi`) e saídas (`MesesParaAtingir, DataPrevistaAlvo, TotalInvestido, TotalRetirado, TotalAcumulado, LucroLiquido, AtingiuMeta, Falta`), além de `Origem`, `StepAtual`, `Versao`, `GeradoEm`.

EvolucaoMensalSimulacao — `Mes, DataReferencia, AporteMensal, AportesExtras, Retiradas, RendimentoBruto, Imposto, RendimentoLiquido, TotalAcumulado, Participantes[] (saldo e rendimento por pessoa)`.

ParametrosFinanceiros — ver §5.5.

Cascata de exclusão

`DeletePlanejamentoCascadeAsync` remove, nesta ordem: gastos detalhados → participantes → aportes extras → histórico de aportes → simulações (e evoluções) → planejamento. `DELETE /api/usuario/{id}` apaga **todos** os planos do usuário e depois o usuário.

---

5. Regras de negócio e motor de cálculo

5.1 Meta (total necessário)

```
valorEntrada   = valorImovel × percentualEntrada / 100
custosExtras   = valorImovel × percentualCustosExtras / 100
totalNecessario = valorEntrada + custosExtras
```

`CustosCompra` também guarda ITBI/escritura/registro calculados pelos percentuais de `ParametrosFinanceiros`, mas **o `TotalNecessario` usado na simulação vem de `percentualCustosExtras`** (valor definido pelo usuário/UI, que sugere esse percentual a partir da tabela de ITBI+cartório, §9.5).

5.2 Simulação de acúmulo (`SimulacaoService.ExecutarSimulacaoAsync`)

Taxa efetiva

```
cenarioDelta   = pessimista −2,0 | realista 0 | otimista +2,0   (pontos percentuais no CDI)
taxaCdiEfetiva = max(0, taxaCDI + cenarioDelta)
taxaAnual      = (taxaCdiEfetiva/100) × (percentualCdi/100)
taxaMensal     = (1 + taxaAnual)^(1/12) − 1
```

`percentualCdi` vem da requisição (se > 0), senão do plano, senão do padrão (100). Os percentuais por tipo de investimento (poupança 70%, CDB 100%, CDB 120% etc.) estão em `ParametrosFinanceiros.Investimentos`.

Saldo inicial

1. Por participante: `ValorAtual` (checkpoint) se existir; senão `PatrimonioInicial.Valor`.
2. Saldo conjunto: `request.ValorJaGuardado` (se > 0) → soma individual (se > 0) → `planejamento.ValorJaGuardado`.
3. Aportes extras/retiradas do **mês 0** (mesmo mês de `DataInicio`) entram no saldo inicial.

Loop mensal (mês 1, 2, ...)

1. Separa aportes extras e retiradas cuja data cai no mês (ignorando revertidos).
2. Aporte regular por pessoa: valor padrão → substituído por edição por pessoa/mês (`aportesRegularesEditadosPorPessoa`) → ou edição legada global proporcional (`aportesRegularesEditados`).
3. **Base de rendimento (pro-rata):**

```
base = saldoInicioMes
     + aporteRegularTotal × (30 − 5)/30          // dia de aporte fixo = 5
     + Σ extra × (30 − diaDoExtra)/30
     − retiradas
```

   (`max(0, base)`)
4. `rendimentoBruto = base × taxaMensal`
5. **IR regressivo** sobre o rendimento do mês (zero se `tipoInvestimento == "lci_lca"`):

Prazo desde o início	Alíquota
até 180 dias	22,5%
até 360 dias	20%
até 720 dias	17,5%
acima	15%

   Os "dias" são aproximados por `mês × 30`. A tabela vem de `AliquotasIr` (parametrizável); se não achar faixa, usa 15%.
6. `rendimentoLiquido = rendimentoBruto − imposto`
7. `novoSaldo = max(0, saldo + aportes + extras − retiradas + rendimentoLiquido)`
8. **Rateio por participante:** o rendimento líquido é distribuído proporcionalmente ao saldo individual (saldo anterior + aportes do mês). Extras sem dono (`extrasGlobais`) são divididos pela mesma proporção.

Critério de parada / horizonte

- Limite inicial: `prazoMaxMeses` do plano; sem prazo, `PrazoFinanciamentoPadraoMeses` (360).
- Ao atingir a meta (`saldo ≥ totalNecessario`) no mês *m*, registra `MesesParaAtingir = m` e ajusta o horizonte para `m + 6` meses (para exibir meses pós-meta):
  - Com prazo definido pelo usuário: `max(prazoOriginal, m + 6)` (**o prazo informado é respeitado**).
  - Sem prazo: `min(limite, m + 6)`.
- Se nunca atinge: `MesesParaAtingir = último mês simulado`, `AtingiuMeta = false`, `Falta = totalNecessario − saldo`.

Resultados: `TotalInvestido` (saldo inicial + aportes + extras − retiradas), `TotalRetirado`, `TotalAcumulado`, `LucroLiquido = saldoFinal − totalInvestido`.

Persistência: apaga as simulações anteriores do plano, grava `HistoricoSimulacao` + `EvolucaoMensalSimulacao[]` e retorna o resultado.

Mapeamento de IDs: o frontend pode enviar IDs temporários (`p1`). O serviço reconcilia com os IDs reais por ID exato → único participante → nome (case-insensitive).

5.3 Financiamento (`FinanciamentoService`)

Taxa mensal: `i = (1 + taxaAnual/100)^(1/12) − 1`.

Operação	Fórmula
SAC	amortização fixa `PV/n`; juros `saldo × i`; parcela = amortização + juros.
Price	`PMT = PV·i·(1+i)^n / ((1+i)^n − 1)`; juros sobre saldo; amortização = PMT − juros.
CET	Fluxo SAC + seguro MIP (`saldo × taxaMip`) + DFI (`PV × taxaDfi`) + taxa de administração; TIR por **Newton-Raphson** (100 iterações, tolerância 1e-6); CET anual = `(1+tirMensal)^12 − 1`, em %.
Comprometimento	Aprovado se `parcela ≤ rendaBrutaFamiliar × LimiteComprometimentoRenda` (padrão **30%**).
FGTS	Modalidade **1**: abate saldo devedor. **3**: desconta `FgtsPercentualParcela` (80%) da parcela por 12 meses. **2** (reduzir prazo): *não implementada* (retorna erro).

Parcelas arredondadas a 2 casas; o último mês zera o saldo devedor.

5.4 Diagnóstico (`CalculoFinanceiroService`)

- `SobraMensal = max(0, renda − gastos)` (no draft: `renda + rendaComplementar − gastos`).
- `CalcularDiagnostico`: sobra da pessoa 1, da pessoa 2, soma do casal e `ValorFaltante = max(0, totalNecessario − (saldoGuardado + aportesExtras))`.

5.5 Parâmetros financeiros (`ParametrosFinanceiros`)

Documento único "ativo" (código `default`), editável por `PUT /api/parametros-financeiros` (**role Admin**). Valores padrão:

Parâmetro	Padrão
`TaxaCdiAnualPadrao`	10,5
`PercentualCdiPadrao`	100
`PrazoMaxSimulacaoMeses`	600
`PrazoFinanciamentoPadraoMeses`	360
`TaxaFinanciamentoAnualPadrao` / MCMV / SBPE	9,5 / 8,16 / 10,5
`LimiteComprometimentoRenda`	0,30
`FgtsPercentualParcela`	0,80
`CustoItbiPadrao` / `CustoEscrituraPadrao` / `CustoRegistroPadrao`	0,02 / 0,01 / 0,005 (fração do valor)
`ItbiIsencaoSp` / `TetoSfh`	335.000 / 1.500.000
`ItbiCheio` / `ItbiSfh` / `CapCustosCompra`	0,03 / 0,005 / 0,04

Tabelas embutidas:

- *Investimentos (% do CDI):* poupança 70, conta corrente 25, CDB/renda fixa 100, CDB 120,

Tesouro Selic 100, LCI/LCA 100, Fundo DI 100, FGTS 50, previdência 90, cripto 125.
- *IR regressivo:* tabela do §5.2.
- *Faixas de cartório:* até 100 mil → R$ 1.500 fixos; até 300 mil → 1,5%; até 700 mil → 1,3%; até 1,5 mi → 1,1%; acima → 0,9%.
- *Bancos de referência (taxa a.a. aproximada):* Caixa 11,19 · BRB 11,36 · Itaú 11,60 · Banco do Brasil 11,60 · Santander 11,69 · Bradesco 11,70.

5.6 Pontos de interesse

`AggregatedPontoInteresseService`:

1. Arredonda lat/lon a 3 casas (~110 m) e ordena categorias → chave de cache (`pontosInteresseCache`, TTL 15 dias).
2. Consulta todos os provedores registrados em paralelo, com timeout por provedor (~8 s nos testes).
3. **Deduplica** itens da mesma categoria a menos de 30 m (Haversine), mesclando tags/fontes.
4. Mantém os **5 mais próximos por categoria**, ordenados por distância.
5. Grava no cache.

Categorias: `mercado, farmacia, escola, padaria, parque, hospital`. Raio máximo: **5.000 m** (padrão 2.000).

---

6. Referência da API REST

Base: `/api`. Autenticação por cookie `token` (JWT) — endpoints marcados 🔒 exigem `[Authorize]`. Erros de validação retornam `400 { message, errors[] }`. Erros não tratados retornam `500 { message, traceId }`.

Saúde e metadados

Método	Rota	Descrição
GET	`/`	`{ service: "ImovPlan API", status: "ok" }`
GET	`/health`, `/api/health`	Health check (inclui ping ao MongoDB, timeout 3 s). Isento de redirecionamento HTTPS.
GET	`/api/version`	Commit (`version.txt`) e `deploy.log`.
GET	`/api/rates/current`	`{ selic, cdi, ipca }` via BrasilAPI. Público.

Auth — `/api/auth` (rate limit `auth`: 10 req/min/IP)

Método	Rota	Corpo	Resposta
POST	`/register`	`{ email, password, name, dataNascimento }`	`{ user }` + cookie
POST	`/login`	`{ email, password }`	`{ user }` + cookie
POST	`/logout`	—	remove cookie
POST	`/google` (limite `google-auth`)	`{ token }` (ID token Google)	`{ user }` + cookie
POST	`/forgot-password`	`{ email }`	envia e-mail com link (30 min,

			uso único)
GET	`/validate-reset-token?token=`	—	`{ message, email }`
POST	`/reset-password`	`{ token, newPassword }`	OK

Usuário 🔒 — `/api/usuario`

Método	Rota	Descrição
GET	`/{id}`	Perfil (somente o próprio usuário).
PUT	`/{id}`	Atualiza `name` e `dataNascimento` (`yyyy-MM-dd`).
PATCH	`/{id}/senha`	`{ senhaAtual, novaSenha }`.
DELETE	`/{id}`	Exclui conta e todos os dados.
GET	`/exportar`	Download JSON com usuário + plano (portabilidade).

Planos (rascunho) 🔒 — `/api/plano`

Método	Rota	Descrição
GET	`/user/{usuarioId}`	Draft do plano mais recente; `204` se não houver. `403` se ID ≠ token.
POST	`/draft-for-user?usuarioId=`	Obtém ou cria o draft do usuário.
GET	`/user/{usuarioId}/todos` · `/todos`	Lista de `PlanoResumoDto`.
POST	`/user/{usuarioId}/novo`	Cria **novo** plano ("Imóvel", "Imóvel 2", …).
GET	`/draft/{id}`	Draft completo (`204` se inexistente/não autorizado).
PUT	`/draft/{id}`	**Autosave** — upsert de objetivo, banco, pessoas, gastos, extras, edições e meses concluídos. `204`.
PATCH	`/{id}/nome`	Renomeia (corpo = string JSON).
POST	`/{id}/link-user?usuarioId=`	Vincula plano órfão (convidado) à conta.
POST	`/{id}/concluir`	Status → `Completed`.
DELETE	`/{id}`	Exclui o plano e dados associados.

Simulação 🔒 — `/api/simulacao`

Método	Rota	Descrição
GET	`/{planoId}/ultima`	Última simulação + evolução mensal (cache em memória 5 min); `204` se não houver.
POST	`/{planoId}/calcular`	Recalcula, persiste e devolve. Corpo: `SimulacaoRequestDto`.

Corpo de `calcular`:

```
{
  "objetivoId": "…",
  "valorJaGuardado": 10000, "valorImovel": 300000,
  "percentualEntrada": 20, "percentualCustosExtras": 5,
  "prazoMaxMeses": 360, "dataInicio": "2025-01-01",
  "aportesMensais": [{ "pessoaId": "…", "pessoaNome": "Ana", "valor": 1500,
                       "valorAtual": 12000, "dataValorAtual": "2025-03-01" }],
  "aportesExtras":  [{ "pessoaId": "…", "valor": 20000, "data": "2025-06-01",
                       "origem": "Bônus", "tipo": "Aporte" }],
  "aportesRegularesEditados": { "3": 2000 },
  "aportesRegularesEditadosPorPessoa": { "<pessoaId>": { "3": 800 } },
  "taxaCDI": 10.5, "percentualCdi": 100, "cenario": "realista"
}
```

A resposta (`SimulacaoResultDto`) traz entradas, saídas, `detalhesMensais[]` (com `participantes[]`) e `participantesSnapshot[]`.

Financiamento 🔒 — `/api/financiamento`

Método	Rota	Corpo
POST	`/simular`	`{ valorFinanciado, taxaAnual, prazoMeses ≤ 1200 }` → `{ SAC, PRICE }`
POST	`/cet`	`+ { taxaMip, taxaDfi, taxaAdmin }` → `{ CetEstimadoAnual }`
POST	`/fgts`	`{ saldoDevedor, saldoFgts, modalidade, parcelaAtual, prazoRestante }`
GET	`/bancos`	Lista de bancos de referência.
POST	`/comprometimento`	`{ rendaBrutaFamiliar, parcelaCalculada }` → `{ Aprovado, ParcelaMaximaPermitida, Diferenca }`

> Respostas de financiamento usam **PascalCase** (objetos anônimos); o frontend normaliza.

Consultoria IA 🔒 — `/api/consultoria`

Método	Rota	Descrição
POST	`/analisar`	Resposta completa `{ text }` (legado).
POST	`/analisar/stream`	**SSE** (`text/event-stream`): linhas `data: <trecho>` (`\n` escapado como `\\n`), finaliza com `data: [DONE]`.

Localização e arredores 🔒

Método	Rota	Descrição
GET	`/api/location/cep/{cep}`	Endereço via BrasilAPI. Limite `geocode`: 1 req/s.
GET	`/api/location/search?q=`	Proxy do Nominatim (geocodificação).
GET	`/api/location/reverse?lat=&lon=`	Proxy do Nominatim (reverso).
GET	`/api/arredores/pontos-interesse?lat&lng&raio&categorias`	Pontos de interesse (§5.6).
POST	`/api/arredores/avaliar-regiao` (limite `ia`:	Avaliação de região por IA a partir de

	10/min)	contagens/locais.

Parâmetros 🔒

Método	Rota	Descrição
GET	`/api/parametros-financeiros`	Parâmetros ativos.
PUT	`/api/parametros-financeiros`	**Admin** — upsert.

CRUD legado 🔒

`/api/planejamento` (GET lista, GET/PUT por id, POST, `GET {id}/diagnostico`, `POST {id}/aportes-extras`) e `/api/participante` (GET, GET/{id}, POST, PUT/{id}, DELETE/{id}). O fluxo principal do frontend usa `/api/plano/draft`; esses endpoints permanecem como API de granularidade fina. Em todos, o plano precisa pertencer ao usuário autenticado.

---

7. Autenticação e segurança

Fluxo de sessão

1. Login/registro/Google gera um **JWT HS256** (claims: `sub`, `jti`, `NameIdentifier`, `Role`; issuer `ImovPlanAPI`, audience `ImovPlanClient`, `ClockSkew = 0`).
2. O token vai em cookie **`token`** — `HttpOnly`, `SameSite=Lax`, `Secure` conforme HTTPS/`X-Forwarded-Proto` (ou `Auth:CookieSecure`). Validade do **cookie**: 7 dias (desktop) ou ~10 anos (UA mobile). Validade do **JWT**: `Jwt:ExpirationInHours` (o `appsettings.json` define **24 h**; sem essa chave o código usaria 168 h); em UA mobile é fixa em ~10 anos.
3. O backend lê o JWT do cookie (`OnMessageReceived`). O frontend guarda apenas um cookie **não sensível** `user` (id, nome, e-mail, nascimento) para hidratação otimista da UI; a validade real é revalidada via `GET /usuario/{id}`.
4. Resposta `401` no frontend → limpa cookie `user` e redireciona para `/auth` (exceto rotas de auth/usuário).

Senhas

- **BCrypt** (work factor 12 na troca de senha).
- Migração transparente de hashes legados (SHA-256 + salt fixo): ao logar com sucesso, regrava em BCrypt.
- Força mínima: ≥ 8 caracteres, com letra e número.
- Cadastro exige idade entre 18 e 120 anos.

Google OAuth

- Valida o ID token (`Google.Apis.Auth`) contra `GOOGLE_CLIENT_ID` e exige `email_verified`.
- Conta local pré-existente com o mesmo e-mail é convertida para `google` e **a senha é descartada** (evita pré-sequestro de conta). Ao definir senha depois via reset, vira `both`.

Recuperação de senha

- Token aleatório de 32 bytes (Base64 URL-safe); no banco fica só o **hash SHA-256**; validade 30 min; uso único; um token válido por vez.

- Link: `{FRONTEND_URL}/auth/reset-password?token=…`.

Autorização

- Todo acesso a plano/simulação/usuário confere o dono (`UsuarioId` do plano == `sub` do token). Respostas `404` (e não `403`) em vários pontos para não revelar existência.
- `LinkPlanoToUser` só vincula planos **sem dono**.
- Atualização de parâmetros financeiros: role `Admin`.

Proteções de infraestrutura

- *Rate limiting* (janela fixa por IP): `auth`, `google-auth`, `ia` = 10/min; `geocode` = 1/s; status 429.
- Cabeçalhos: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, `HSTS` (exceto localhost).
- CORS restrito a `CORS_ALLOWED_ORIGINS` (com credenciais).
- Frontend: **CSP** em produção, `X-Frame-Options`, `poweredByHeader` desligado.
- Validação com **FluentValidation** em filtro global (`ValidationFilterAttribute`).
- Sanitização do prompt de avaliação de região (máx. 10 locais, 60 caracteres, sem quebras de linha).
- Contêiner de produção roda como usuário não root (`USER app`).

---

8. Integrações externas

Serviço	Uso	Configuração	Observações
MongoDB Atlas	Persistência	`MONGO_CONNECTION_STRING`, `MONGO_DATABASE_NAME`	Health check no `/health`.
Groq (`llama-3.3-70b-versatile`, temp. 0,4)	Consultoria financeira e avaliação de região	`GROQ_API_KEY`	Sem chave válida usa modo **mock**. Timeout 30 s (60 s no stream). O prompt injeta simulação SAC/Price reais, limite de 30% e faixas do MCMV; resposta em Markdown.
Brevo	E-mail transacional (reset de senha)	`BREVO_API_KEY`, `BREVO_FROM_EMAIL`	Provedor **registrado** no DI.
Resend / SMTP	E-mail alternativo	—	Implementações existem mas **não estão registradas**.
BrasilAPI	Taxas (`/taxas/v1`: Selic, CDI, IPCA) e CEP	—	Em falha, taxa retorna `0`.
Overpass (OpenStreetMap)	Pontos de interesse	—	Único provedor registrado.
SerpApi (Google Local)	Pontos de interesse (alternativo)	`SERPAPI_KEY`	Implementado, não registrado.
Nominatim	Busca/reverso de endereço	User-Agent próprio	Respeitar 1 req/s (já limitado).

Google Identity	Login social	`GOOGLE_CLIENT_ID` (backend) / `NEXT_PUBLIC_GOOGLE_CLIENT_ID` (frontend)	`@react-oauth/google`.

9. Frontend

9.1 Stack

Next.js 16 (App Router, dev com `--webpack`) · React 19 · TypeScript · Tailwind CSS v4 · Zustand 5 (persistido) · TanStack Query 5 · Axios · Recharts 3 · react-markdown · react-virtuoso · sonner · lucide-react · date-fns · use-debounce · Vitest + Testing Library + jsdom.

> `frontend/AGENTS.md` alerta que esta versão do Next.js tem mudanças incompatíveis; consulte `node_modules/next/dist/docs/` antes de alterar convenções.

9.2 Rotas

Rota	Função
`/`	Landing page pública (SEO, OG image, sitemap, robots).
`/auth`, `/auth/forgot-password`, `/auth/reset-password`	Login, cadastro, Google, recuperação.
`/termos`, `/termos-de-uso`, `/privacidade`, `/politica-de-privacidade`, `/version`	Páginas legais e versão.
`/app/onboarding`	Define cenário padrão `entrada` e redireciona.
`/app/imovel`	**Passo 1** — dados do imóvel (valor, entrada, ITBI/cartório por cidade, prazo, CDI, investimento).
`/app/pessoas`	**Passo 2** — participantes, rendas, gastos detalhados, aporte, patrimônio.
`/app/planejamento`	**Passo 3** — "Acelere seu plano": aportes extras/retiradas, cenários, viabilidade.
`/app/resultado`	Resultado: cards, gráfico, tabela mês a mês, consultoria IA.
`/app/lancar-aporte`	Lançar aporte do mês / marcar mês concluído / atualizar saldo real.
`/app/pronto`	Financiamento de imóvel pronto (SAC×Price, regra dos 30%, concluir plano).
`/app/planta`	Fluxo de pagamentos de imóvel na planta (eventos, calendário).
`/app/financiamento`	Simulação do banco escolhido.
`/app/planos`	Meus planos: criar, renomear, excluir, comparar.
`/app/perfil`	Dados pessoais, redefinição de senha, exportar/excluir conta.

O `AppLayout` exige autenticação (redireciona para `/auth`), ajusta `document.title` e mostra

skeleton durante o carregamento. A navegação (`AppShell`) varia por cenário: *entrada* (Imóvel → Pessoas → Plano → Resultado), *pronto* (Financiamento → Plano → Resultado), *planta* (Fluxo → Plano → Resultado).

9.3 Gerenciamento de estado e dados

- **`usePlanStore` (Zustand + persist):** `planoId`, `objetivo`, `pessoas`, `aportesExtras`, `bancoEscolhido`, `cenario` (`entrada|pronto|planta`), `cenarioSimulacao` (`pessimista|realista|otimista`), edições de aporte, `mesesConcluidos`, dia do aporte e *hashes* do último payload calculado/salvo (para evitar chamadas redundantes). Trocar `planoId` reinicia o estado.
- **`usePlanDraft`:** `GET /plano/draft/{id}` (ou `/plano/user/{id}`), mapeia para o store; em erros 5xx/rede **aborta** (não sobrescreve dados). `useSaveDraft`: `PUT` (ou cria via `draft-for-user` e depois `PUT`); `404` aborta o autosave para não ressuscitar plano apagado em outra aba.
- **`usePlanLogic`:** orquestra tudo — carrega draft e última simulação, expõe `saveDraft` (**debounce 1 s**) e `calcularBackend`, calcula progresso atual (saldo do dia via evolução diária, ou meses concluídos). O recálculo no backend também roda com debounce (~1,2 s) após mudanças.
- **React Query:** chaves `planDraft`, `planos`, `simulacao`; `staleTime` 5 min; sem refetch em foco.
- **`AuthContext`:** register/login/google/forgot/reset/logout/delete; hidratação otimista + revalidação em segundo plano; limpa Zustand e cookies no logout; emite o evento `imovplan:auth-changed`.

9.4 Camada de serviços

`api.ts` (axios, `withCredentials`, interceptor 401) e `PlanoService`, `SimulacaoService`, `UsuarioService`, `ConsultoriaService` (inclui leitor de **SSE** com `fetch` + `ReadableStream`), `FinanciamentoService`, `ExternalDataService` (taxas e CEP).

9.5 Utilitários de domínio

- **`lib/finance.ts` (≈ 820 linhas, puro/testável):** `simular()`, `aliquotaIR`, taxas mensal/diária (252 dias úteis), evolução diária, `calcularEvolucaoDiariaDoMes`, `mesesParaMeta`, `aporteNecessarioParaPrazo` (busca binária), sugestões (aporte adicional, novo prazo, redução do imóvel), mapeamento de tipo de investimento → % do CDI. Espelha o motor C# (teste de paridade) e é usado para sugestões e visões diárias.
- **`utils/itbi.ts`:** tabela de ITBI por UF/cidade (SP isento ≤ R$ 335 mil, 0,5% SFH até R$ 1,5 mi, 3% acima; RJ 3%; DF isento ≤ R$ 200 mil; demais 1–3%, padrão 2%) + cartório escalonado → custo total e `descricaoRegra`.
- **`utils/simulacaoSelectors.ts`:** extrai saldo atual, resumo, participantes e série do gráfico a partir do resultado do backend (O(n)).
- **`utils/dates.ts`:** datas no fuso de Brasília; `ibge-estados-cidades.ts`: seletor de UF/cidade.

9.6 PWA, SEO e segurança

- `manifest.json` (`start_url /app/imovel`, tema `#E8590C`), `sw.js` (v8): navegação *network-first* com fallback `/offline.html`; `_next/static` *cache-first*; demais *stale-while-revalidate*; **`/api/*` nunca é interceptado**.
- `lib/seo.ts`, `sitemap.ts`, `robots.ts`, `opengraph-image.tsx`.

- `next.config.ts`: CSP, headers e *rewrites* `/api/* → NEXT_PUBLIC_API_URL`.

---

10. Configuração e variáveis de ambiente

Backend (`backend/.env` ou ambiente)

Variável	Obrig.	Descrição
`JWT_SECRET`	✅	≥ 32 bytes. A API **não sobe** sem ela.
`JWT_ISSUER` / `JWT_AUDIENCE`		Padrões `ImovPlanAPI` / `ImovPlanClient`.
`Jwt:ExpirationInHours`		`appsettings.json` define 24 h; sem a chave, o código assume 168 h. Mobile: 87.600 h fixas.
`MONGO_CONNECTION_STRING`	✅	String do Atlas.
`MONGO_DATABASE_NAME`	✅	Ex.: `ImovPlanDb`.
`CORS_ALLOWED_ORIGINS`	✅	URLs do frontend separadas por vírgula, sem barra final.
`FRONTEND_URL`	✅	Base dos links de e-mail.
`GOOGLE_CLIENT_ID`	se Google	Valida o ID token.
`GROQ_API_KEY`	se IA	Sem ela, IA em modo mock.
`BREVO_API_KEY`, `BREVO_FROM_EMAIL`	se e-mail	Envio de reset de senha.
`SERPAPI_KEY`	opcional	Provedor SerpApi (não registrado hoje).
`PORT`		Definida por Vercel/PaaS; mapeada para `ASPNETCORE_URLS`.
`Auth:CookieSecure`		Força flag `Secure` do cookie.
`KEEP_ALIVE_URL`, `REDIS_CONNECTION_STRING`, `ASPNETCORE_ALLOWED_HOSTS`		Constam no `.env.example`; não vi uso no código (ver §14).

O `.env` é carregado por **DotNetEnv** (procura em vários caminhos relativos) e depois por variáveis de ambiente.

Frontend (`frontend/.env.local`)

Variável	Descrição
`NEXT_PUBLIC_API_URL`	URL do backend (com ou sem `/api`; o rewrite normaliza). Padrão `http://localhost:5179`.
`NEXT_PUBLIC_APP_URL`	URL pública do site (SEO/compartilhamento).
`NEXT_PUBLIC_GOOGLE_CLIENT_ID`	Client ID do Google.

---

11. Execução local, build e deploy

Pré-requisitos

.NET SDK 9, Node 20+, MongoDB (Atlas ou local), Docker (opcional).

Local sem Docker

```
# Backend
cd backend
cp .env.example .env            # preencha JWT_SECRET, MONGO_*, CORS_ALLOWED_ORIGINS, FRONTEND_URL
dotnet run --project Presentation/ImovPlan.API   # Swagger em /swagger (Development)

# Frontend
cd frontend
cp .env.example .env.local
npm ci
npm run dev                                      # http://localhost:3000
```

O `launchSettings.json` do projeto API define a porta de desenvolvimento; o `docker-compose` usa **5179**.

Docker Compose

```
export JWT_SECRET="…" MONGO_CONNECTION_STRING="…" MONGO_DATABASE_NAME="ImovPlanDb" GOOGLE_CLIENT_I
docker compose up --build
# backend:  http://localhost:5179   (container 8080, limite de memória 350 MB)
# frontend: http://localhost:3000   (build standalone do Next)
```

O compose monta `version.txt` e `deploy.log` (somente leitura) para o endpoint `/api/version` e um volume para chaves do Data Protection. **Atenção:** `./deploy.log` deve existir no host, senão o Docker cria um diretório no lugar.

Vercel (monorepo)

`vercel.json` na raiz declara dois serviços: `web` (`frontend`, Next.js) e `backend` (`backend/Dockerfile.vercel`), com *rewrites* de `/api/*` para o backend e o resto para o web. O backend na Vercel escuta em `PORT`, tem `HEALTHCHECK` em `/health` e **não** precisa de `KEEP_ALIVE_URL`. Passos (de `backend/README.md`):

```
cd backend
dotnet build Presentation/ImovPlan.API/ImovPlan.API.csproj -c Release
vercel deploy          # preview
vercel deploy --prod
```

Cadastre as variáveis do §10 em Preview e Production e confira `https://<api>/health`.

---

12. Testes

Backend — xUnit + Moq (`net10.0`)

Arquivo	Cobertura
`CalculoFinanceiroServiceTests`	Entrada, sobra e diagnóstico (0/1/2 participantes, falta negativa → 0).
`FinanciamentoServiceTests`	SAC/Price: soma das amortizações = PV e saldo final 0; caso conhecido SAC.

`SimulacaoServiceTests`	Prazo respeitado/limite, fallback sem prazo, meta antes do prazo, saldo global sem titular.
`SimulacaoParidadeTests`	Paridade com o motor do frontend (fixtures).
`PlanejamentoServiceDraftTests`	Upsert de draft, campos legados nulos em aportes extras.
`PlanoControllerTests`	Autorização e retornos do `UpdateDraft`.
`Services/*`	Cache, timeout e deduplicação de pontos de interesse; categorização Overpass; execução paralela por categoria no SerpApi.

```
cd backend && dotnet test
```

Frontend — Vitest

`lib/finance.test.ts`, `utils/dates.test.ts`, `utils/simulacaoSelectors.test.ts`.

```
cd frontend && npm test
```

Paridade front × back

`scripts/gerar-casos-paridade.ts` roda `simular()` sobre cenários fixos (básico, prazo curto com bônus, sem prazo definido) e escreve `tests/fixtures/casos-paridade.json`, consumido pelo `SimulacaoParidadeTests`. **Regenere as fixtures sempre que alterar qualquer regra do motor** em um dos lados.

---

13. Fluxos principais (ponta a ponta)

1. Criar e calcular um plano

```
/app/imovel  ── salva objetivo ─► PUT /plano/draft/{id} (debounce 1 s)
/app/pessoas ── salva pessoas ──► PUT /plano/draft/{id}
               └─ dispara cálculo ► POST /simulacao/{id}/calcular
/app/planejamento ─ extras/edições ► PUT draft → recálculo (debounce ~1,2 s)
/app/resultado ── GET /simulacao/{id}/ultima (cache 5 min) → gráficos e tabela
```

2. Lançar aporte do mês — o usuário marca o mês como concluído (`mesesConcluidos`) e/ou informa o saldo real (`valorAtual` / `dataValorAtual`). O backend usa esse checkpoint no lugar do saldo teórico e recalcula as projeções futuras.

3. Financiamento — `/app/pronto` chama `POST /financiamento/simular`, compara SAC×Price, aplica a regra dos 30% (`/comprometimento`) e permite concluir o plano (`POST /plano/{id}/concluir`).

4. Consultoria IA — o front envia perfil + imóvel; o backend monta o contexto com simulação real SAC/Price e transmite o laudo em Markdown via SSE; o front concatena os trechos até `[DONE]`.

5. Convidado → conta — um plano criado sem dono pode ser vinculado após o login por `POST /plano/{id}/link-user`.

6. Exclusão de conta (LGPD) — `GET /usuario/exportar` baixa os dados; `DELETE /usuario/{id}` remove todos os planos em cascata e o usuário.

14. Divergências e pontos de atenção

Itens observados na leitura do código; úteis como backlog técnico.

Divergências com o README

1. O README cita *Refresh Tokens* e *confirmação de e-mail*; **não encontrei** implementação de nenhum dos dois — há um único JWT em cookie e apenas e-mail de reset de senha.
2. O README referencia `LICENSE` (MIT), mas o arquivo **não existe** no repositório.
3. O README lista `.NET 9`, mas o projeto de testes usa **`net10.0`** (e `EF Core InMemory 10.x`).
4. O README cita `contexts/` e Axios em `lib/`; na prática a pasta é `context/` e o estado principal é o **Zustand**.

Comportamento do código 5. **Dia do aporte fixo:** o motor usa `diaReg = 5` para o pro-rata do aporte regular; `Participante.DiaAporte` é salvo mas **não influencia** o cálculo do backend. 6. **IR aproximado:** alíquota é escolhida por `mês × 30` desde o início do plano e aplicada ao rendimento de cada mês (modelo simplificado, sem FIFO por aporte — assumido no próprio código). 7. **FGTS modalidade 2** (reduzir prazo) retorna "não suportada". 8. **`LembretePlanejamentoService`** roda a cada 24 h mas é **apenas simulação** (loga, não envia e-mail). 9. **SerpApi, Resend e SMTP** têm implementação, porém só **Overpass** e **Brevo** estão registrados no DI. 10. **`REDIS_CONNECTION_STRING`, `KEEP_ALIVE_URL` e `ASPNETCORE_ALLOWED_HOSTS`** aparecem em `.env.example`/docs, sem uso aparente no código. 11. **`/auth/forgot-password` retorna 404** quando o e-mail não existe — permite **enumeração de contas** (o cadastro, ao contrário, usa mensagem genérica). 12. **`UpdateDraft` expõe `ex.Message`** no corpo do 500 (`detail`) — considere ocultar em produção. 13. **Cookie mobile de ~10 anos** (JWT de 87.600 h) amplia a janela de risco em caso de vazamento do token; não há revogação. 14. **Descompasso cookie × JWT no desktop:** o cookie dura 7 dias, mas o JWT expira em 24 h (`appsettings.json`). Passado esse prazo o cookie ainda existe, o token não vale e a API responde 401 (o frontend então redireciona para `/auth`). Além disso, `Key`/`Issuer`/`Audience` da seção `JWT` do `appsettings.json` são ignorados: o código lê `JWT_SECRET`, `JWT_ISSUER` e `JWT_AUDIENCE`. 15. **`PlanejamentoController` (legado)** aceita a entidade `Planejamento` direta no corpo (risco de *overposting*, ex.: `ParticipantesIds`, `Status`) — o fluxo atual usa DTOs de draft. 16. **`ParametrosFinanceiros` default:** em `ParametrosFinanceirosController.Put` o documento recebido é salvo integralmente por upsert; garanta validação se expor a mais perfis administrativos. 17. **`Class1.cs`** vazio em `Infrastructure` (resquício de template). 18. **Base de custos duplicada:** ITBI/cartório existem em três lugares (`ParametrosFinanceiros`, `CustosCompra` e `utils/itbi.ts`); o total da simulação usa o `percentualCustosExtras` — mantenha as tabelas sincronizadas. 19. **Locks de draft em memória** (`ConcurrentDictionary<string, SemaphoreSlim>`): só serializam gravações dentro de **uma instância**; com várias réplicas, a concorrência no mesmo plano não é protegida. 20. **Cache de última simulação em memória (5 min)** é por instância e pode servir dado defasado após alterações feitas em outra réplica.

15. Glossário

Termo	Significado
Aporte	Valor mensal guardado. **Aporte extra**: injeção pontual (FGTS, 13º...). **Retirada**: saída do

	saldo.
Checkpoint	Saldo real informado pelo usuário (`ValorAtual` + `DataValorAtual`) que substitui o teórico.
CDI	Taxa de referência do rendimento da renda fixa; `% do CDI` define o ganho do investimento.
CET	Custo Efetivo Total do financiamento (juros + seguros + taxas), expresso a.a.
DFI / MIP	Seguros do financiamento (danos físicos ao imóvel / morte e invalidez permanente).
ITBI	Imposto de Transmissão de Bens Imóveis (municipal).
MCMV / SBPE	Minha Casa Minha Vida / Sistema Brasileiro de Poupança e Empréstimo.
SAC / Price	Sistemas de amortização: parcelas decrescentes / parcelas fixas.
SFH	Sistema Financeiro da Habitação (alíquota reduzida de ITBI em alguns municípios).
INCC	Índice Nacional de Custo da Construção (correção de parcelas na planta).
Draft	Rascunho do plano, salvo automaticamente; vira `Completed` ao concluir.